A BM TECHNICAL MANUAL

The bml Language

The typed subset of Lua for the Pi Zero W: syntax, stack VM, kernel bmvm, and native ARMv6 A32 code with C and ASM equivalents.

Contents

Why a language of bm 3

One source, four targets 3

Syntax and type rules 4

Types and literals 4

Statements and operators 4

How the compiler works 6

Stage 1 and Stage 2: AST and stack bytecode 6

Stage 3 and Stage 4: Kernel `bmvm` and ARMv6 A32 6

Practical code examples: BML, C, and ARMv6 ASM 7

Example 1: Loop accumulator (`sum`) 7

Example 2: Value clamping (`clamp`) 7

Example 3: Euclidean greatest common divisor (`gcd`) 8

Example 4: Double recursion (`fib`) 9

Example 5: Structs, arrays, and VFPv2 `f32` (`step_vec2f`) 10

Example 6: Game cartridge optimization (Dark Night) 11

Try it step by step 13

Open points 14

README

What this is. How to write hot functions in the typed language of bm (`tools/bml/bml.py`), run the same source in plain Lua, and inspect the generated stack bytecode, kernel `bmvm` register code, C equivalent, and native 32-bit ARMv6 A32 machine instructions for the Raspberry Pi Zero W.

How to read it. Every rule, opcode, and machine word comes from `tools/bml/bml.py` , `scripts/bmt.py` , `src/bm/bmvm.h` , and `docs/design/bml-manual.md` . Chapter 4 shows four complete functions with their BML source, C equivalent, and disassembled ARMv6 assembly side by side.

What is missing. Every open stage of the roadmap in `docs/progress/typed.md` is marked with a highlighted TO FILL tag and listed in the last chapter.

Why a language of bm

A game on the Raspberry Pi Zero W 1.1 runs on a single-core ARM1176JZF-S at 1000 MHz with a frame budget of 16.7 ms at 60 fps (`docs/PERFORMANCE.md`). When the Lua profiler (`profile(true)`) shows a hot function called thousands of times per frame, you want native speed without maintaining two separate codebases.

One source, four targets

A `.b` file uses the syntax of Lua with type annotations (`: i32` , `: bool`) on parameters and return values. From a single `.b` file, `tools/bml/bml.py` gives:

Target	Flag / API	Where it runs	Purpose
Plain Lua 5.4	`--lua`	`luahost` and the console Lua VM	Reference oracle: strips `: <type>` and `struct ... end`
Stack VM (Stage 2)	`--run / --dump`	Host Python (`StackVM`)	Fast verification and bytecode inspection
Kernel `bmvm` (Stage 3)	`--bmvm`	`src/bm/bmvm.c` (`vm_load` , `vm_run`)	Safe register-machine blob inside a cartridge
Native ARMv6 A32 (Stages 4–8)	`--armv6 / --native`	`src/bm/bmnative.c` (`native_load` , `native_run`)	Native 32-bit A32 + VFPv2 words in executable RAM (AAPCS)
On-Console Compiler (Stage 9)	`native_compile(src)`	`src/bm/bmlc.c` (`bml_compile_native`)	Compiles `.b` source directly to `BMN1` ARMv6 machine code on the Pi

NOTE

`scripts/bmt.py` translates the same typed Lua syntax to C (`--c`) for functions linked into the kernel (`src/bm/api_collmap.c`). `tools/bml/bml.py` is the standalone compiler that generates bytecode and ARMv6 machine words directly without a C compiler.

Syntax and type rules

The front end (`Lexer` , `Parser` , `AST` , `TypeChecker` in `tools/bml/bml.py`) checks every statement before emitting code. Any error stops compilation and prints `line <N>: <reason>` .

Types and literals

Type	Meaning	Literals / Syntax	ARMv6 representation
`i32`	Signed 32-bit integer (2's complement wrap)	`0` , `42` , `-2147483648` , `2147483647`	32-bit general-purpose register (`r0` .. `r10`)
`bool`	Strict truth value	`true` , `false`	`1` (`true`) or `0` (`false`) in a 32-bit register
`f32`	IEEE-754 single-precision float	`0.0` , `1.5` , `-0.25`	32-bit GP slot + VFPv2 coprocessor 10 `s0` .. `s1`
`i32[]` , `bool[]` , `f32[]`	1-based typed arrays	`a[i]` , `a[i] = v`	Base pointer in 32-bit register (`ldr / str` at `base + (i - 1) * 4`)
`struct Name ... end`	Fixed-layout record	`s.field` , `s.field = v`	Pointer to contiguous 32-bit field words (`offset = field_idx * 4`)

- Conditions in `if` , `elseif` , `while` , `and` , `or` , and `not` must have type `bool` . An `i32` is never implicitly truthy.
- Integer division uses `//` and modulo uses `%` (floor toward $-\infty$). Float division uses `/` on `f32` operands.
- Built-in math and conversion functions: `abs(x)` , `min(a, b)` , `max(a, b)` (for `i32` or `f32`), `sqrt(x: f32): f32` , `f32(x: i32): f32` , and `floor(x: f32): i32` .
- Kernel host functions (Stage 7+): `cls(col)` , `pix(x, y, col)` , `rectfill(x, y, w, h, col)` , `spr(id, x, y)` , `btn(b): bool` , `btnp(b): bool` , `line(x0, y0, x1, y1, col)` , `circ(x, y, r, col)` , `circfill(x, y, r, col)` , `rect(x, y, w, h, col)` , `sfx(id)` , `music(id)` , and `time(): f32` .

Statements and operators

Construct	Example	Rule
Function	`function add(a: i32, b: i32): i32 ... end`	Up to 16 parameters; supports recursion and calls to earlier functions
Local variable	`local s = 0` or `local s: i32 = 0`	Scoped to its enclosing block; cannot shadow an active variable

Construct	Example	Rule
Assignment	`s = s + i`	Target must be declared and have the exact same type
Branch	`if a < b then ... elseif a == b then ... else ... end`	Every condition must be `bool`
While loop	`while b ~= 0 do ... end`	Condition must be `bool`; `break` exits the innermost loop
Numeric for	`for i = 1, n do ... end` or `for i = n, 1, -2 do ... end`	Bounds and optional step are `i32`
Operators	`+`, `-`, `*`, `//`, `%`, `&`, `<`, `<=`, `>`, `>=`, `==`, `~=`, `and`, `or`, `not`	`and` and `or` short-circuit as in Lua

How the compiler works

Stage 1 and Stage 2: AST and stack bytecode

1. `tokenize()` scans the source into `Token(kind, text, line, start, end)` records.
2. `Parser` builds the `Program` syntax tree (`FuncDecl` , `Stmt` , `Expr`) and records the byte spans of `: i32` and `: bool` so `to_lua()` can strip them cleanly.
3. `TypeChecker` verifies all variable scopes, operator types, function calls, and checks that every path of a value-returning function ends with `return` .
4. `BytecodeCompiler` emits stack instructions (`PUSH_CONST` , `LOAD` , `STORE` , `ADD` , `SUB` , `MUL` , `IDIV` , `IMOD` , `BAND` , `LE` , `JZ` , `JF_KEEP` , `JT_KEEP` , `CALL` , `RET`).

Stage 3 and Stage 4: Kernel `bmvm` and ARMv6 A32

1. `compute_stack_depths()` computes the exact height of the evaluation stack before every instruction.
2. `BMVMCompiler` maps parameters to `R[0..nparams-1]` , constants to `R[nparams..nparams+nconst-1]` , locals to `R[nparams+nconst..]` , and stack slots `S[k]` above locals, emitting a `BMV1` blob for `src/bm/bmvm.c` .
3. `ARMv6Compiler` emits 32-bit ARMv6 A32 machine words following the AAPCS ABI (`r0` .. `r3` arguments, `r0` return value, `r4` .. `r10` callee-saved registers holding the first 7 local/stack slots, and `[fp, #-off]` stack slots for spills).
4. `compile_native_blob()` (`--native`) packs the `BMN1` module for `src/bm/bmnative.c` (`native_load` , `native_fn` , `native_run`): `bmnative_load` validates the A32 instructions, copies them into 32-byte-aligned executable RAM, cleans the D-cache (`dcache_clean_range`), invalidates the I-cache (`mcr p15, 0, a, c7, c5, 1`) and branch target cache (`c7, c5, 6`), and executes `DSB` + `ISB` .

Practical code examples: BML, C, and ARMv6 ASM

Example 1: Loop accumulator (`sum`)

The canonical loop from `tests/bml/sum.b` sums the integers from `1` to `n` .

```
-- 1. BML source (tests/bml/sum.b)
function sum(n: i32): i32
  local s = 0
  for i = 1, n do s = s + i end
  return s
end
```

```
/* 2. Equivalent C (scripts/bmt.py --c) */
int32_t bmt_sum(int32_t n)
{
    int32_t s = 0;
    {
        int32_t i__end = n;
        for (int32_t i = 1; i <= i__end; i++) {
            s = (s + i);
        }
    }
    return s;
}
```

```
; 3. ARMv6 A32 leaf machine code (9 words = 36 bytes, r0 = n, r1 = s, r2 = i)
  0x0000: E3A01000    mov   r1, #0          ; s = 0
  0x0004: E3A02001    mov   r2, #1          ; i = 1
  0x0008: E1520000    cmp   r2, r0          ; compare i with n
  0x000c: CA000002    bgt   0x1c            ; if i > n jump to 0x1c
  0x0010: E0811002    add   r1, r1, r2      ; s = s + i
  0x0014: E2822001    add   r2, r2, #1      ; i = i + 1
  0x0018: EAFFFFFA    b     0x08            ; repeat loop
  0x001c: E1A00001    mov   r0, r1          ; return s in r0
  0x0020: E12FFF1E    bx    lr              ; return to caller
```

Example 2: Value clamping (`clamp`)

`clamp` restricts `x` to the `[lo, hi]` interval using the built-in `min` and `max` intrinsics, which lower to conditional moves (`movlt` , `movgt`) on ARMv6 without branches.

```
-- 1. BML source (tests/bml/features.b)
function clamp(x: i32, lo: i32, hi: i32): i32
  return min(max(x, lo), hi)
end
```

```
/* 2. Equivalent C */
int32_t bmt_clamp(int32_t x, int32_t lo, int32_t hi)
{
    return bmt_min_i32(bmt_max_i32(x, lo), hi);
}
```

```
; 3. ARMv6 A32 machine code (r4 = x, r5 = lo, r6 = hi, r7/r8 = stack slots)
  0x004c: E92D4FF0    push  {r4-r10, fp, lr}
  0x0050: E1A0B00D    mov   fp, sp
  0x0054: E3A0C004    mov   ip, #4
  0x0058: E04DD00C    sub   sp, sp, ip
  0x005c: E1A04000    mov   r4, r0          ; r4 = x
  0x0060: E1A05001    mov   r5, r1          ; r5 = lo
  0x0064: E1A06002    mov   r6, r2          ; r6 = hi
  0x0068: E1A07004    mov   r7, r4          ; S[0] = x
  0x006c: E1A08005    mov   r8, r5          ; S[1] = lo
  0x0070: E1570008    cmp   r7, r8          ; compare x with lo
  0x0074: B1A07008    movlt r7, r8          ; max(x, lo): if x < lo then r7 = lo
  0x0078: E1A08006    mov   r8, r6          ; S[1] = hi
  0x007c: E1570008    cmp   r7, r8          ; compare max(x, lo) with hi
  0x0080: C1A07008    movgt r7, r8          ; min(..., hi): if r7 > hi then r7 = hi
  0x0084: E1A00007    mov   r0, r7          ; result in r0
  0x008c: E1A0D00B    mov   sp, fp
  0x0090: E8BD8FF0    pop   {r4-r10, fp, pc}
```

Example 3: Euclidean greatest common divisor (`gcd`)

`gcd` uses `abs` , a `while` loop, and `%` . Because the ARM1176 has no hardware integer division instruction, `a % b` calls the 30-word A32 helper `__bml_idivmod` , which returns the floor quotient in `r0` and the floor modulo in `r1` .

```
-- 1. BML source (tests/bml/features.b)
function gcd(a: i32, b: i32): i32
  a = abs(a)
  b = abs(b)
  while b ~= 0 do
    local r = a % b
    a = b
    b = r
  end
  return a
end
```

```
/* 2. Equivalent C */
int32_t bmt_gcd(int32_t a, int32_t b)
{
    a = abs(a);
    b = abs(b);
    while ((b != 0)) {
        int32_t r = bmt_imod(a, b);
        a = b;
        b = r;
    }
    return a;
}
```

```
; 3. ARMv6 A32 machine code (r4 = a, r5 = b, r6 = r)
  0x00ac: E1A07004    mov   r7, r4
  0x00b0: E3570000    cmp   r7, #0
  0x00b4: B2677000    rsblt r7, r7, #0      ; abs(a) via reverse-subtract if < 0
  0x00b8: E1A04007    mov   r4, r7
  ...
; while b ~= 0 loop body:
  0x00f0: E1A00007    mov   r0, r7          ; r0 = a
  0x00f4: E1A01008    mov   r1, r8          ; r1 = b
  0x00f8: EB0001B5    bl    __bml_idivmod   ; r0 = a // b, r1 = a % b
  0x00fc: E1A07001    mov   r7, r1          ; S[0] = remainder r1
  0x0100: E1A06007    mov   r6, r7          ; local r = a % b
  0x0104: E1A07005    mov   r7, r5
  0x0108: E1A04007    mov   r4, r7          ; a = b
  0x010c: E1A07006    mov   r7, r6
  0x0110: E1A05007    mov   r5, r7          ; b = r
  0x0114: EAFFFFEC    b     0x00cc          ; jump back to while test
```

Example 4: Double recursion (`fib`)

In `fib(n - 1) + fib(n - 2)` , the first recursive call places its result into `S[0]` , which is mapped to callee-saved register `r5` . When the second `bl fn_fib` runs, `fn_fib` saves and restores `r4` .. `r10` on its stack frame, so `r4` (`n`) and `r5` (`fib(n - 1)`) remain intact without extra memory traffic.

```
-- 1. BML source (tests/bml/features.b)
function fib(n: i32): i32
  if n <= 0 then
    return 0
  elseif n == 1 then
    return 1
  else
    return fib(n - 1) + fib(n - 2)
  end
end
```

```
/* 2. Equivalent C */
int32_t bmt_fib(int32_t n)
{
    if ((n <= 0)) return 0;
    else if ((n == 1)) return 1;
    else return (bmt_fib((n - 1)) + bmt_fib((n - 2)));
}
```

```
; 3. ARMv6 A32 recursive else branch (r4 = n, r5 = S[0], r6 = S[1])
  0x0198: E1A05004    mov   r5, r4          ; S[0] = n
  0x019c: E3A06001    mov   r6, #1          ; S[1] = 1
  0x01a0: E0455006    sub   r5, r5, r6      ; S[0] = n - 1
  0x01a4: E1A00005    mov   r0, r5          ; arg0 = n - 1
  0x01a8: EBFFFFDF    bl    fn_fib          ; call fib(n - 1)
  0x01ac: E1A05000    mov   r5, r0          ; r5 (callee-saved) = fib(n - 1)
  0x01b0: E1A06004    mov   r6, r4          ; r6 = n (r4 preserved across call)
  0x01b4: E3A07002    mov   r7, #2          ; r7 = 2
  0x01b8: E0466007    sub   r6, r6, r7      ; r6 = n - 2
  0x01bc: E1A00006    mov   r0, r6          ; arg0 = n - 2
  0x01c0: EBFFFFD9    bl    fn_fib          ; call fib(n - 2) (r5 preserved)
  0x01c4: E1A06000    mov   r6, r0          ; r6 = fib(n - 2)
  0x01c8: E0855006    add   r5, r5, r6      ; r5 = fib(n - 1) + fib(n - 2)
  0x01cc: E1A00005    mov   r0, r5          ; return result in r0
```

Example 5: Structs, arrays, and VFPv2 `f32` (`step_vec2f`)

Stage 6 lowers `f32` arithmetic (`+` , `-` , `*` , `/` , `sqrt` , comparisons) to hardware VFPv2 coprocessor 10 instructions (`vadd.f32` , `vmul.f32` , `vsqrt.f32` , `vcmp.f32` + `vmrs` APSR_nzcv, fpscr) and compiles `struct` field accesses (`v.x` , `v.y`) to word loads and stores.

```
-- 1. BML source (tests/bml/types_mem.b)
struct Vec2f
  x: f32
  y: f32
end

function step_vec2f(v: Vec2f, dt: f32, lim: f32): i32
  v.x = v.x + 2.0 * dt
  v.y = v.y - 1.5 * dt
  local d: f32 = sqrt(v.x * v.x + v.y * v.y)
  if d > lim then
    v.x = v.x * (lim / d)
    v.y = v.y * (lim / d)
  end
  return floor(d * 100.0)
end
```

```
/* 2. Equivalent C */
typedef struct { float x, y; } Vec2f;

int32_t step_vec2f(Vec2f *v, float dt, float lim)
{
    v->x = v->x + 2.0f * dt;
    v->y = v->y - 1.5f * dt;
    float d = sqrtf(v->x * v->x + v->y * v->y);
    if (d > lim) {
        v->x = v->x * (lim / d);
        v->y = v->y * (lim / d);
    }
    return (int32_t)floorf(d * 100.0f);
}
```

```
; 3. ARMv6 + VFPv2 coprocessor 10 machine code (distance & comparison)
  EE007A10    vmov.f32  s0, r7          ; s0 = v.x * v.x + v.y * v.y
  EEB10AC0    vsqrt.f32 s0, s0          ; hardware VFPv2 square root
  EE107A10    vmov.f32  r7, s0          ; r7 = d (IEEE-754 binary32 bits)
  EE007A10    vmov.f32  s0, r7          ; s0 = d
  EE006A90    vmov.f32  s1, r6          ; s1 = lim
  EEB40A60    vcmp.f32  s0, s1          ; compare d with lim in VFPv2
  EEF1FA10    vmrs      APSR_nzcv, fpscr; transfer VFP flags to ARM CPSR
```

Example 6: Game cartridge optimization (Dark Night)

In `carts/darknight/main.lua`, enemy separation checks 272 pairs every frame ($O(N^2)$). In Lua, table lookups (`e.x` , `o.x` , `o.st`) and floating-point VM operations take 27.3% of the total logic time. Converting the loop to a `.b` hot function with `vm_buf` eliminates table boxing, uses hardware VFPv2 single-precision instructions, and reduces runtime from 20 us to 4 us (up to 5x faster).

```
-- 1. Original Lua in carts/darknight/main.lua
for i = 1, #enemies do
  local o = enemies[i]
  local ox = ex - o.x
  if ox < 0.8 and ox > -0.8 and o ~= e and o.st ~= "dead" then
    local oz = ez - o.z
    local d2 = ox * ox + oz * oz
    if d2 < 0.64 and d2 > 0.000001 then
      try_move(e, ox / sqrt(d2) * 0.02, oz / sqrt(d2) * 0.02, 0.35)
    end
  end
end
```

```
-- 2. Typed .b implementation (tests/bml/darknight_enemy.b)
function move_and_separate(n: i32, x: f32[], z: f32[], vx: f32[], vz: f32[], alive: bool[], out: i32[],
  for self = 1, n do
    if alive[self] then
      for i = 1, n do
        if i ~= self and alive[i] then
          local dx = x[self] - x[i]
          if dx < 0.8 and dx > -0.8 then
            local dz = z[self] - z[i]
            local d2 = dx * dx + dz * dz
            if d2 < 0.64 and d2 > 0.00001 then
              counts[self] = counts[self] + 1
              out[(self - 1) * n + counts[self]] = i
            end
          end
        end
      end
    end
  end
  return 0
end
```

```
-- 3. Zero-copy integration in cart with native_compile and vm_buf
local blob = assert(native_compile(b_source))
local mod = assert(native_load(blob))
local bx = vm_buf("i32", N)
local bz = vm_buf("i32", N)
native_run(mod, "move_and_separate", N, bx, bz, bvx, bvz, balive, bout, bcounts)
```

Implementation	17 enemies scan	Calls per frame	Crossing cost
Original Lua tables	15 - 20 us	0 (Lua VM)	Table hashing
Multiple native calls	8 - 12 us	34 calls	34 us overhead
Single call + vm_buf	3 - 4 us	1 call	1 us total

Try it step by step

TRY IT

Compile `tests/bml/sum.b` and `tests/bml/darknight_enemy.b`, inspect the stack bytecode and ARMv6 + VFPv2 machine words, or compile `.b` source strings directly in Lua with `native_compile(src)`:

1. Run `python3 tools/bml/bml.py tests/bml/sum.b --dump --run sum 10`: you see the stack instructions and the result `55`. 2. Run `python3 tools/bml/bml.py tests/bml/sum.b --dump-armv6`: you see the 9 hexadecimal A32 words from `0x0000` to `0x0020`. 3. Run `make test-bml`: the suite checks 23 invalid programs, `sum.b`, `features.b`, 200 random programs, `types_mem.b`, `kernel_fn.b`, `darknight_enemy.b` (60 frames of 17 enemies), and the Stage 9 on-console C compiler `native_compile` across Lua, `StackVM`, C `bmvm`, `native_run`, and `qemu-system-arm -cpu arm1176`.

COPY AND PASTE

```
python3 tools/bml/bml.py tests/bml/sum.b --dump --run sum 10
python3 tools/bml/bml.py tests/bml/sum.b --dump-armv6
python3 tools/bml/bml.py tests/bml/features.b --native build/features.bmn
make test-bml
```

Open points

No open points.